

Trabalho 1 de MC458 - PxNP

Antônio Queiroz Manetta - RA:231565

21 de Maio de 2023

1 Introdução

O seguinte projeto tem por objetivo fornecer a solução para o problema do Parque Exótico Nacional dos Picos (PxNP). Neste problema, é dado que existe um parque com montes e tirolesas conectando cada par de montes. Porém, cada tirolesa só pode ser utilizada em um sentido. Pede-se para determinar o maior caminho possível, ou seja, percorrer o maior número de montes possível, sem passar por um mesmo monte duas vezes.

Este problema pode ser modelado pela estrutura de grafos, ou seja, um conjunto de vértices e um conjunto de arestas interligando os vértices. Neste caso específico, o grafo é dito direcionado, pois todas as arestas podem ser usadas apenas em um dos dois sentidos. Também é dito que o grafo é completo, visto que existe uma aresta entre quaisquer pares de vértices, em uma das duas direções. Assim, fica modelado o problema e podemos construir um programa que resolva esse problema. O código tem por objetivo retornar um vetor que indica o caminho a ser trilhado, desde o vértice inicial, passando todos os pontos, e chegando ao último, sem repetir nenhum vértice.

Porém, queremos que o problema seja resolvido em menos de 1 segundo. Estima-se que uma máquina faça em torno de 10^7 operações em 1 segundo. Sabendo que a entrada do problema será no máximo $N = 10^5$, o programa deve resolver este problema e a complexidade de tempo do algoritmo deve ser menor que $O(N^2)$, podendo ser um algoritmo com complexidade $O(N \log(N))$, dependendo da base do logaritmo. Assim, vamos buscar um algoritmo recursivo para resolver este problema.

2 A corretude do algoritmo

De posse do grafo, podemos observar que sempre existirá um caminho que liga todos os vértices, sem passar mais de uma vez por algum deles. Isto é, seguindo as restrições de completude e direcionamento, qualquer exemplo de grafo de tamanho N sempre terá uma solução que contemple cada um dos N vértices. Assim, podemos teorizar sobre como achar esse caminho, e como construir o algoritmo que acha esse caminho rapidamente.

Estamos partindo do pressuposto que sempre haverá um caminho que liga todos os vértices, logo, para qualquer vértice que escolhermos aleatoriamente, ela sempre estará em alguma posição da lista que representa o caminho trilhado. Seja o vértice inicial v_0 , podemos repartir o problema em dois subproblemas iguais da seguinte maneira: para o vértice v_0 , todos os outros vértices ou chegam a v_0 por meio de uma única aresta, ou uma única aresta sai de v_0 chegando aos vértices subsequentes. Estes dois subconjuntos de vértices que, ou chegam ou saem de v_0 , serão chamados respectivamente de vértices predecessores de v_0 e posteriores de v_0 . Assim, para solucionar o problema inicial, basta solucionarmos os dois subproblemas de vértices predecessores e posteriores. Ou seja, ordenar os vértices predecessores e posteriores, colocando o vértice v_0 no meio das duas listas. Para ordenarmos os dois subconjuntos, escolhemos novamente um dos vértices para separar o subconjunto em mais dois subconjuntos, e assim resolvemos recursivamente o problema, até os casos base: se para algum vértice, não existem predecessores ou posteriores, ou existe apenas um predecessor ou posterior, não há mais o que ordenar. No

caso de não existir nenhum vértice, apenas retornamos uma lista vazia, e no caso de apenas um vértice, retornamos uma lista com apenas esse vértice. Após a resolução dos casos base, concatenamos a lista de predecessores com o vértice intermediário e com a lista de posteriores, e assim por diante até que retornemos ao caso inicial, concatenando a lista de predecessores, o vértice v_0 , e a lista de posteriores.

Feita esta análise prévia, podemos demonstrar matematicamente os fatos. Seja o conjunto V de vértices, escolher um vértice v . Para o resto dos vértices, seja C o conjunto dos vértices que chegam em v e S o conjunto de vértices que saem de v . Sejam C_s a solução para o conjunto C (ou seja, a solução para o problema inicial mas com o conjunto C) e S_s a solução para o conjunto S . A solução para o conjunto V , V_s , é dada pela concatenação da lista C_s , do elemento v e da lista S_s : $V_s = C_s + [v] + S_s$. Assim, sempre obteremos soluções para o conjunto V .

Observe que, para isto funcionar, precisamos destacar o fato de que para qualquer nível de recursão que estivermos, qualquer vértice do conjunto C chega em v e v chega em qualquer vértice do conjunto S . Além disso, como excluimos o vértice v dos dois subproblemas, sabemos que a solução nunca passará duas vezes pelo mesmo vértice. Portanto, a solução sempre será válida.

3 A Implementação do Código

Dada a intenção do código e o panorama geral, podemos agora explicar como se dá a implementação.

Primeiramente, criamos a função `solve ( int N )`, que recebe um único parâmetro N , que indica o número de vértices do grafo. Essa função nada mais faz do que particionar o caso inicial usando um pivô aleatório e depois chamar a função `solve_recursivo ( int l_sai )` e `solve_recursivo ( int l_entra )` e atribuir os retornos às listas $L1$ e $L2$, respectivamente. Depois, concatena as duas listas com o pivô selecionado como explicado anteriormente. O processo de partição se resolve criando dois laços percorrendo os vértices do problema exceto o pivô, um laço para os vértices com índice menores que o pivô, e outro para vértices com índice maior que o pivô. Para cada um deles, usasse a função `has_edge` para determinar em qual das listas cada um dos vértices estará presente, na lista `l_sai` ou na lista `l_entra`.

Na função `solve_recursivo` que realmente aplicaremos o conceito da recursão. Basicamente, esta função fará a mesma coisa que a função `solve`, apenas com as diferenças que ela tem como entrada uma lista, e depois de, com base nessa lista, separarmos os vértices, chamamos a função `solve_recursivo` recursivamente. Além disso, essa função traz a solução para o caso base, que é quando a lista de entrada é vazia ou contém apenas um elemento. Em ambos os casos, apenas retornamos a própria lista.

Como o tempo para achar a solução é dependente do pivô, além disso, usaremos um inicializador aleatório para o pivô a cada passo da recursão. Explicado o código, podemos seguir para a complexidade de tempo, e também explicar o por que de uma escolha de um pivô aleatório.

4 Complexidade Temporal do Código

O algoritmo como um todo tem maior similaridade com o algoritmo Quick-Sort Aleatório. Basicamente, o algoritmo faz exatamente o mesmo processo, só que para um problema diferente. Em vez de ordenar um vetor em ordem crescente, queremos ordenar um vetor em uma ordem tal que existam arestas ligando-os nessa ordem e nesse sentido. Assim, a ordenação é dada numa forma recursiva igual a do Quick-Sort, exceto pelo fato de que para cada nova partição dos subproblemas chamaremos a função `has_edge` para cada um dos vértices dos subconjuntos, e enfim particioná-los em novos subconjuntos. Dessa forma, o código toma $O(N)$ para inicializar as variáveis, depois chama duas recursões, e depois retorna o resultado tomando $O(N)$. Assim, podemos escrever a seguinte recorrência:

$$T(n) = T(n - k) + T(k - 1) + O(n) \quad (1)$$

Esta recorrência é similar a recorrência do algoritmo Quick-Sort, e assim como esta, aquela nos dá no pior caso um resultado $T(n) \in O(n^2)$. Também sabemos que no melhor caso e no caso médio, que é mais próximo do melhor caso, esta recorrência tem um resultado $T(n) \in O(n \log n)$. Assim, foi utilizado um inicializador aleatório para cada pivô utilizado em cada recorrência, dessa forma o número esperado de comparações também é $T(n) \in O(n \log n)$. O pivô escolhido aleatoriamente nos garante que não existirá casos em que o algoritmo nunca passará no teste, mas que em todos os testes a serem feitos o algoritmo tem uma grande probabilidade de passar no tempo esperado.

A prova do resultado da recorrência anterior é similar a do Quick-Sort aleatório. Porém, essa prova é complexa e envolve cálculo de probabilidades. Portanto, podemos nos contentar que o resultado será o mesmo.

5 Conclusão

Visto que a complexidade de tempo deste algoritmo é esperada ser $O(n \log n)$, submetemos o código e obtemos resultados corretos e dentro do tempo esperado para todos os 20 testes. Isso indica que ao aleatorizar a escolha do pivô, será extremamente improvável nos depararmos com um tempo de execução perto do pior caso, e portanto o código é válido para resolver testes aleatórios com $N = 10^5$, e assim justifica-se a escolha do algoritmo.